TRƯỜNG ĐẠI HỌC SÀI GÒN (SGU) • KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO TOÀN DIỆN PHẦN 1: MÔ HÌNH SẢN PHẨM & ĐA HÌNH BẢO HÀNH

ĐỀ TÀI: QUẢN LÝ CỬA HÀNG THỜI TRANG (OOP JAVA) • PHẦN 1

Phân công: Thành viên 1 | GVHD: Thầy Trần Ngọc Minh Hiếu (tnmhieu@sgu.edu.vn)

TÀI LIỆU CHI TIẾT DÀNH CHO THÀNH VIÊN HỌC & BẢO VỆ VẤN ĐÁP (ĐIỂM 10/10)

> **Phân công phụ trách:** Thành viên 1

> **Mục tiêu:** Hiểu sâu từng dòng code, phân biệt rõ code chính - code phụ trợ, nắm chắc kỹ thuật chính - kỹ thuật phụ và trả lời trôi chảy mọi câu hỏi vấn đáp của Thầy Hiếu.

1. DANH MỤC CÁC TẬP TIN (FILES) QUẢN LÝ TRONG PHẦN 1

Phần 1 gồm tổng cộng **9 tập tin mã nguồn (.java)**:

1.1. Các tập tin mã nguồn chính (Core Files) - 7 Lớp

1. `SanPhamThoiTrang.java` : Lớp cha trừu tượng (Abstract Base Class), đóng vai trò xương sống cho toàn bộ mô hình hàng hóa.
2. `AoKhoac.java` : Lớp con chuyên biệt quản lý áo khoác (có/không có nón, bảo hành 12 tháng).
3. `AoThun.java` : Lớp con chuyên biệt quản lý áo thun (kích cỡ S, M, L, XL).
4. `QuanJean.java` : Lớp con chuyên biệt quản lý quần jean (kiểu dáng Slim, Baggy, Ống rộng).
5. `PhuKien.java` : Lớp con chuyên biệt quản lý phụ kiện (chất liệu da, kim loại, bảo hành 6 tháng).
6. `VayDam.java` : Lớp con chuyên biệt quản lý váy đầm thời trang nữ (chất liệu vải voan, lụa, ren).
7. `GiayDep.java` : Lớp con chuyên biệt quản lý giày dép (size số từ 35 đến 45, bảo hành 3 tháng).

1.2. Các tập tin phụ trợ & Giao tiếp (Supporting & Interfaces) - 2 Interface

8. `IBaoHanh.java` : Interface định nghĩa năng lực bảo hành chính hãng (`printWarrantyCard` , `getWarrantyPeriod`).
9. `IGiamGia.java` : Interface định nghĩa khả năng chiết khấu giá sản phẩm (`calculateAfterDiscount`).

2. BẢNG PHÂN LOẠI KĨ THUẬT LẬP TRÌNH: CHÍNH VS PHỤ TRỢ

Phân loại	Tên kĩ thuật	Vị trí xuất hiện trong mã nguồn	Vai trò & Ý nghĩa kỹ thuật
Kỹ thuật chính	**Tính Trừu tượng (Abstraction)**	`abstract class SanPhamThoiTrang` , hàm `getThongTinRieng()` , `getStringSaveFile()`	Tạo khuôn mẫu chuẩn, ngăn khởi tạo đối tượng chung chung bằng lệnh `new` , bắt buộc lớp con phải cụ thể hóa.
Kỹ thuật chính	**Tính Kế thừa (Inheritance)**	`extends SanPhamThoiTrang` , `super(Type, MaSP, TenSP,`	Tái sử dụng 100% thuộc tính chung của sản phẩm, hình

		`GiaSP)`	thành cây phân cấp danh mục hàng hóa.
Kỹ thuật chính	**Tính Đa hình (Polymorphism)**	`@Override` các phương thức trừu tượng và Interface `IBaoHanh`	Tại runtime, cùng lời gọi hàm nhưng mỗi lớp con tự thực hiện hành vi riêng biệt (Áo khoác in 12 tháng, Giày in 3 tháng).
Kỹ thuật chính	**Tính Đóng gói (Encapsulation)**	`private` fields, các getter/setter có validation	Che giấu trạng thái nội tại, kiểm soát giá bán không âm, chuẩn hóa dữ liệu hợp lệ.
Kỹ thuật chính	**Hiện thực Interface**	`implements IBaoHanh` , `implements IGiamGia`	Biểu diễn quan hệ "CAN-DO" (năng lực bảo hành). Tách rời hành vi bảo hành ra khỏi bản chất sản phẩm.
Kỹ thuật phụ trợ	**Thành phần Static (Tĩnh)**	`public static final TI_LE_VAT` , `tongSoLuongSanPhamDaTao`	Hằng số dùng chung toàn hệ thống (10% VAT) và biến đếm tổng đối tượng cấp lớp chia sẻ 1 vùng nhớ duy nhất.
Kỹ thuật phụ trợ	**Tiện ích hiển thị Console**	`formatTien()` , `formatCell()` , `inTieuDeBang()` , `inChanBang()`	Canh lề, đóng khung viền bảng như Excel, cắt chuỗi tự động (`...`) chống vỡ bố cục khi tên quá dài.
Kỹ thuật phụ trợ	**Làm sạch dữ liệu (Sanitization)**	`.replace(",", " ")` , `.toUpperCase()`	Khử dấu phẩy chống vỡ định dạng CSV khi lưu file text, chuẩn hóa size áo hoa thường.

3. PHÂN TÍCH CHI TIẾT TỪNG ĐOẠN CODE: CHÍNH VS PHỤ TRỢ

3.1. Tại tập tin `SanPhamThoiTrang.java`

Đoạn Code A: Khai báo lớp trừu tượng và thuộc tính Static

```
public abstract class SanPhamThoiTrang implements IGiamGia {
    public static final double TI_LE_VAT = 0.10;
    private static int tongSoLuongSanPhamDaTao = 0;

    private String Type;
    private String MaSP;
    private String TenSP;
    private double GiaSP;
```

- **Kỹ thuật chính:**
- `public abstract class` : Khai báo lớp trừu tượng. Lớp này không thể khởi tạo thực thể trực tiếp (`new SanPhamThoiTrang()` sẽ báo lỗi biên dịch).
- `private ...` : Đóng gói an toàn toàn bộ thuộc tính, ngăn chặn việc gán sai từ lớp khác.

- **Kỹ thuật phụ trợ:**
- `public static final double TI_LE_VAT = 0.10` : Hằng số tĩnh đại diện 10% thuế VAT. Không bị nhân bản bộ nhớ theo từng đối tượng.
- `private static int tongSoLuongSanPhamDaTao = 0` : Biến tĩnh cấp lớp để theo dõi số lượng sản phẩm được tạo trong vòng đời ứng dụng.

Đoạn Code B: Hàm khởi tạo (Constructor) có tham số

```
public SanPhamThoiTrang(String Type, String MaSP, String TenSP, double GiaSP) {
        this.Type = (Type != null) ? Type : "ChuaPhanLoai";
        this.MaSP = (MaSP != null) ? MaSP : "";
        this.TenSP = (TenSP != null) ? TenSP : "";
        this.GiaSP = (GiaSP >= 0) ? GiaSP : 0;
        tongSoLuongSanPhamDaTao++;
    }
```

- **Kỹ thuật chính:** Constructor có tham số giúp thiết lập trạng thái khởi tạo cho đối tượng ngay khi cấp phát vùng nhớ.
- **Kỹ thuật phụ trợ (Phòng ngừa lỗi runtime):**
- Toán tử 3 ngôi `(GiaSP >= 0) ? GiaSP : 0` : Ràng buộc đơn giá không được âm.
- `(Type != null) ...` : Khử giá trị `null` thành chuỗi rỗng để chống ngoại lệ `NullPointerException` .
- `tongSoLuongSanPhamDaTao++` : Tự động tăng biến đếm tĩnh mỗi khi có đối tượng mới được tạo.

Đoạn Code C: Phương thức trừu tượng (Abstract Methods)

```
public abstract String getStringSaveFile();
    public abstract String getThongTinRieng();
```

- **Kỹ thuật chính:** Khai báo phương thức trừu tượng không có thân hàm `{}` . Đây là "hợp đồng bắt buộc": Mọi lớp con kế thừa (`AoKhoac` , `AoThun` ...) **bắt buộc phải override** 2 hàm này, nếu không chương trình sẽ không thể biên dịch.

Đoạn Code D: Kỹ thuật phụ trợ định dạng hiển thị Console

```
public static String formatTien(double amount) {
        DecimalFormatSymbols symbols = new DecimalFormatSymbols();
        symbols.setGroupingSeparator('.');
        DecimalFormat df = new DecimalFormat("#,##0", symbols);
        return df.format(Math.round(amount)) + " VND";
    }

    public static String formatCell(String text, int maxLen) {
        if (text == null) return "";
        if (maxLen <= 3) return (text.length() > maxLen) ? text.substring(0, maxLen) : text;
        if (text.length() > maxLen) {
            return text.substring(0, maxLen - 3) + "...";
        }
        return text;
    }
```

- **Kỹ thuật phụ trợ:**
- `formatTien` : Dùng `DecimalFormat` của Java để chuyển đổi `1800000` thành `1.800.000 VND` (chuẩn hiển thị tiền tệ Việt Nam).

- `formatCell` : Hàm cắt chuỗi thông minh. Nếu tên sản phẩm dài hơn độ rộng cột (ví dụ 22 ký tự), hàm tự động cắt và thêm dấu `...` , giúp bảng danh sách trên Console luôn thẳng hàng tuyệt đối.

3.2. Tại tập tin lớp con `AoKhoac.java`

```
public class AoKhoac extends SanPhamThoiTrang implements IBaoHanh {
    private boolean hasHat;

    public AoKhoac(String Type, String MaSP, String TenSP, double GiaSP, boolean hasHat) {
        super(Type, MaSP, TenSP, GiaSP);
        this.hasHat = hasHat;
    }

    @Override
    public String getThongTinRieng() {
        return "Co non: " + (this.hasHat ? "Co" : "Khong");
    }

    @Override
    public void printWarrantyCard() {
        System.out.println("+------------------------------------------------------------+");
        System.out.println("|                    PHIEU BAO HANH                          |");
        System.out.println("+------------------------------------------------------------+");
        System.out.printf("| Ten san pham      : %-33s |\n", getTenSP());
        System.out.printf("| Ma san pham       : %-33s |\n", getMaSP());
        System.out.printf("| Thoi han bao hanh : %-33s |\n", getWarrantyPeriod() + " thang");
        System.out.println("+------------------------------------------------------------+");
    }

    @Override
    public int getWarrantyPeriod() {
        return 12;
    }

    @Override
    public String getStringSaveFile() {
        return getType() + "," + getMaSP() + "," + getTenSP() + "," + getGiaBan() + "," + this.hasHat;
    }
}
```

- **Kỹ thuật chính (Kế thừa & Constructor Chaining):**
- `extends SanPhamThoiTrang` : Thiết lập quan hệ kế thừa.
- `super(Type, MaSP, TenSP, GiaSP)` : Lời gọi constructor lớp cha. Dòng này **bắt buộc phải đứng đầu tiên** trong constructor lớp con để đảm bảo phần thuộc tính kế thừa được khởi tạo trước.
- `this.hasHat = hasHat` : Khởi tạo thuộc tính riêng của áo khoác.
- **Kỹ thuật chính (Đa hình qua Ghi đè phương thức):**
- `@Override getThongTinRieng()` : Hiện thực phương thức trừu tượng từ lớp cha.
- `@Override getWarrantyPeriod()` : Trả về thời hạn bảo hành 12 tháng từ interface `IBaoHanh` .
- `@Override printWarrantyCard()` : Tự đóng khung in phiếu bảo hành riêng cho Áo khoác.
- **Kỹ thuật phụ trợ:**
- `@Override getStringSaveFile()` : Xuất định dạng 5 cột nối bằng dấu phẩy để phục vụ việc ghi vào file `products.txt` .

3.3. So sánh logic các lớp con chuyên biệt còn lại

10. **`AoThun.java`**:

- *Code chính:* `private String size;`
- *Code phụ trợ:* Trong setter `setSize(String size)`, kiểm tra nếu chuỗi nhập vào là `s, m, l, xl` thì dùng `.toUpperCase()` đổi thành chữ hoa. Nếu nhập sai, ép về kích thước mặc định `"M"`.

11. **`QuanJean.java`**:

- *Code chính:* `private String Style;` (Kiểu dáng: Skinny, Baggy, Ống rộng).
- *Code phụ trợ:* Tự động gọi `.replace(",", " ")` để làm sạch chuỗi kiểu dáng khi người dùng nhập có dấu phẩy.

12. **`PhuKien.java`**:

- *Code chính:* `implements IBaoHanh`, `getWarrantyPeriod() { return 6; }` (Bảo hành 6 tháng).

13. **`VayDam.java`**:

- *Code chính:* `private String chatLieuVai;` (Chất liệu vải: Voan, Lụa, Ren).

14. **`GiayDep.java`**:

- *Code chính:* `private int sizeGiay;`, `implements IBaoHanh`, `getWarrantyPeriod() { return 3; }` (Bảo hành 3 tháng).
- *Code phụ trợ:* Kiểm tra điều kiện `sizeGiay >= 35 && sizeGiay <= 45`. Nếu nằm ngoài khoảng sẽ từ chối hoặc gán size chuẩn 40.

4. BỘ CÂU HỎI VẤN ĐÁP CỦA THẦY HIẾU (KÈM DÒNG CODE VÀ CÂU TRẢ LỜI ĐIỂM 10)

CÂU HỎI VẤN ĐÁP: Thầy chỉ vào dòng `public abstract class SanPhamThoiTrang` và hỏi: "Tại sao không để class bình thường mà phải thêm từ khóa `abstract`? Nếu thầy gọi `new SanPhamThoiTrang()` thì chuyện gì xảy ra?"

TRẢ LỜI MẪU (10 ĐIỂM): "Dạ thưa Thầy:
1. `SanPhamThoiTrang` là khái niệm trừu tượng, đại diện chung cho mọi mặt hàng thời trang. Trong thực tế bán hàng, khách hàng chỉ mua 'Áo khoác', 'Áo thun', hoặc 'Giày dép' cụ thể chứ không ai mua một 'sản phẩm thời trang' chung chung. Vì vậy lớp này phải là `abstract` để làm khuôn mẫu chuẩn cho các lớp con kế thừa.
2. Nếu Thầy gọi `new SanPhamThoiTrang(...)`, trình biên dịch Java sẽ **báo lỗi ngay tại thời điểm biên dịch (Compile Error)**, vì Java tuyệt đối ngăn chặn việc khởi tạo thực thể từ một lớp trừu tượng."

CÂU HỎI VẤN ĐÁP: Thầy chỉ vào hàm tạo của `AoKhoac` và hỏi: "Từ khóa `super(Type, MaSP, TenSP, GiaSP)` có tác dụng gì? Đổi vị trí xuống dòng thứ 2 có được không?"

TRẢ LỜI MẪU (10 ĐIỂM): "Dạ thưa Thầy:
1. Từ khóa `super(...)` dùng để gọi trực tiếp constructor 4 tham số của lớp cha `SanPhamThoiTrang`, chuyển giao các giá trị `Type`, `MaSP`, `TenSP`, `GiaSP` lên cho lớp cha khởi tạo và bảo vệ dữ liệu.
2. Lệnh `super(...)` **bắt buộc phải nằm ở dòng lệnh đầu tiên** của constructor lớp con. Nếu đổi xuống dòng thứ 2 hoặc phía sau, trình biên dịch Java sẽ báo lỗi cú pháp ngay lập tức."

CÂU HỎI VẤN ĐÁP: Thầy hỏi: "Tại sao `AoKhoac`, `PhuKien`, `GiayDep` lại implements `IBaoHanh`, còn `AoThun`, `QuanJean` thì không?"

TRẢ LỜI MẪU (10 ĐIỂM): "Dạ thưa Thầy, đây là việc áp dụng đúng nguyên lý thiết kế Hướng đối tượng:

1. Interface thể hiện quan hệ **CAN-DO** (khả năng hành vi). Không phải mọi sản phẩm may mặc đều có chế độ bảo hành. Quần áo thông thường như áo thun, quần jean chỉ áp dụng đổi trả lúc mua chứ không bảo hành theo tháng.

2. Chỉ những mặt hàng có giá trị hoặc phụ kiện/giày dép mới có bảo hành chính hãng. Nhóm thiết kế `IBaoHanh` độc lập để chỉ những lớp nào cần bảo hành mới implement, đảm bảo tính phân tách giao diện (Interface Segregation Principle).

3. Khi xuất hóa đơn, hệ thống chỉ cần dùng toán tử `sp instanceof IBaoHanh` để tự động in phiếu bảo hành đa hình mà không làm ảnh hưởng đến các sản phẩm khác."

CÂU HỎI VẤN ĐÁP: Thầy chỉ vào `public static final double TI_LE_VAT = 0.10;` và hỏi: "Ba từ khóa `public`, `static`, `final` ở đây có ý nghĩa gì?"

TRẢ LỜI MẪU (10 ĐIỂM): "Dạ thưa Thầy:

- `public`: Cho phép mọi lớp khác trong dự án (như `ChiTietHoaDon`, `HoaDon`) có thể truy cập để tính thuế.
- `static`: Biến thuộc về mức lớp (Class-level), chỉ lưu trữ duy nhất 1 vùng nhớ chia sẻ cho toàn bộ ứng dụng, truy cập trực tiếp qua `SanPhamThoiTrang.TI_LE_VAT` mà không cần tạo đối tượng.
- `final`: Biến này là hằng số (Constant), giá trị 0.10 cố định không thể bị gán đè hay thay đổi trong suốt quá trình chạy."

CÂU HỎI VẤN ĐÁP: Thầy chỉ vào dòng `this.TenSP = TenSP.trim().replace(",", " ");` và hỏi: "Lệnh `.replace(",", " ")` để làm gì, nếu không có thì có lỗi gì không?"

TRẢ LỜI MẪU (10 ĐIỂM): "Dạ thưa Thầy, đây là kỹ thuật làm sạch dữ liệu (Data Sanitization):

1. Hệ thống của nhóm lưu danh mục sản phẩm vào file text `products.txt` theo định dạng CSV (phân cách bằng dấu phẩy `,`).

2. Nếu người dùng nhập tên sản phẩm chứa dấu phẩy (ví dụ: 'Áo gió, 2 lớp, nam'), khi đọc lại file bằng lệnh `line.split(",")`, dòng đó sẽ bị tách thành nhiều hơn 5 phần tử, làm lệch các cột giá tiền và thuộc tính riêng, dẫn đến lỗi ngoại lệ đọc file.

3. Lệnh `.replace(",", " ")` chuyển dấu phẩy thành khoảng trắng, triệt tiêu 100% nguy cơ vỡ cấu trúc file text."